SELF-HEALING INFRASTRUCTURE

Project Report

1. Introduction

Self-healing infrastructure is an operational approach in which a system detects a failure and automatically attempts a predefined recovery action. This project demonstrates a local container-based web service monitored with Prometheus and Blackbox Exporter. When the NGINX endpoint becomes unavailable, an alert is routed through Alertmanager to a Python webhook, which invokes an Ansible playbook to restart the stopped container.

2. Abstract

The project implements an automated monitoring and recovery workflow in an Ubuntu virtual machine using Docker. Prometheus checks the HTTP endpoint through Blackbox Exporter and evaluates an alert rule named **NginxDown**. Alertmanager forwards firing alerts to a Python webhook. The webhook runs an Ansible playbook that checks the container state and starts NGINX if it is stopped. During testing, the NGINX container was stopped to simulate failure; webhook logs reported successful playbook execution, the container returned to the running state, and the health probe subsequently reported success.

3. Tools Used

- **Ubuntu Linux:** Project environment inside VirtualBox.
- **Docker:** Runs NGINX and the monitoring components in containers.
- **Prometheus:** Collects metrics and evaluates alerting rules.
- **Blackbox Exporter:** Checks HTTP availability of the NGINX endpoint.
- **Alertmanager:** Routes Prometheus alerts to the webhook.
- **Python:** Receives alert notifications through an HTTP webhook.
- **Ansible:** Runs the recovery playbook to start a stopped NGINX container.

4. Steps Involved in Building the Project

1. **Environment setup:** Prepared an Ubuntu virtual machine and installed Docker and Ansible.
2. **Container deployment:** Deployed NGINX and connected monitoring services through a Docker network.
3. **Health monitoring:** Configured Prometheus to use Blackbox Exporter to test the NGINX HTTP endpoint.
4. **Alert configuration:** Created the **NginxDown** rule, which fires when the probe fails for 30 seconds, and configured Alertmanager to forward alerts to the webhook.
5. **Recovery automation:** Built a Python webhook to receive firing alerts and execute an Ansible playbook. The playbook checks container state and starts NGINX if it is stopped.
6. **Validation:** Stopped NGINX to simulate failure. The webhook logged successful playbook execution, NGINX returned to a running state, and the Prometheus health probe returned **1**.

5. Conclusion

The project successfully demonstrates a basic self-healing workflow by combining service monitoring, alert routing, and automated recovery. It provides practical experience with Docker, Prometheus, Blackbox Exporter, Alertmanager, Python, and Ansible. Before production use, the webhook should be protected with authentication, restricted to trusted network access, and limited to approved recovery actions.

Prepared by: Karthigai Selvan | Environment: Ubuntu Linux on VirtualBox | Project: Self-Healing Infrastructure